

Cross-Head Fusion for Multi-Head Attention Cluster Telemetry Monitoring

Project Report

MSc Research Project / (Research) Practicum / Internship
Cloud Computing

Mehak

Student ID: TBD

School of Computing
National College of Ireland

Supervisor: TBD

September 2026

Cross-Head Fusion for Multi-Head Attention Cluster Telemetry Monitoring

Mehak
TBD

Abstract

Cluster health monitoring traditionally relies on reactive threshold rules that only flag a problem after a metric has already crossed its limit. Recent work by Thapliyal (2026) proposes a proactive alternative for data-center SLA monitoring: a per-entity transformer with one attention head dedicated to each telemetry metric, predicting violations ahead of time. That paper’s own results show a specific weakness – the most volatile metric’s head systematically underpredicts severity during high-load transients, because heads never exchange information with one another. This project reproduces that strict one-head-per-metric architecture as a baseline for cluster telemetry (CPU, memory, disk, network), confirms the same underprediction pattern on synthetic multivariate telemetry with injected cross-metric burst precursors, and evaluates a cross-head fusion layer that lets the per-metric heads attend to each other before classification. Across five training seeds, the fusion model gives a small, directionally consistent improvement on the targeted failure mode – transient violation recall rises from 86.4% to 88.4% and mean underprediction bias falls from 0.29 to 0.27 severity levels – at a marginal cost to overall accuracy (87.3% to 86.3%), with neither difference exceeding one standard deviation across seeds. Both attention-based models substantially outperform reactive threshold monitoring at anticipating violations before they occur (86-88% vs. 2.5% recall on future violations inside transient windows), confirming the baseline paper’s central premise transfers to cluster telemetry, while the fusion improvement itself is real but modest.

1 Introduction

As cloud clusters scale, operators need to anticipate node-level failures before they happen rather than react to them after the fact. Traditional monitoring applies fixed thresholds to individual metrics (CPU, memory, disk I/O, network) and only raises an alert once a metric has already crossed its limit, giving operators no lead time to intervene.

Thapliyal Thapliyal (2026) recently proposed a proactive alternative for the related problem of data-center SLA compliance monitoring: a per-entity multi-head transformer in which each attention head is dedicated to one telemetry rule, predicting violations up to 30 minutes ahead. The paper reports a specific limitation of this design – the most volatile metric’s head systematically underpredicts severity during high-load transients, which the authors attribute to heads never sharing information with one another despite the underlying metrics being correlated.

Research question: does allowing per-metric attention heads to exchange information via a cross-head fusion layer reduce this systematic underprediction during high-load transients, without degrading overall prediction accuracy?

Objectives:

- Reproduce Thapliyal’s strict one-head-per-metric architecture as a baseline, adapted from SLA rules to cluster telemetry metrics (CPU, memory, disk, network).
- Construct a synthetic telemetry benchmark with a genuine forecasting task (history window in, future window labelled) and cross-metric burst precursors, so that predicting a metric’s future violation sometimes requires reading a *different* metric’s early signal.
- Confirm the underprediction pattern reported in the baseline paper on this benchmark.
- Design and evaluate a cross-head fusion layer as a targeted fix, iterating on the training setup (class weighting, epochs, burst design) until a measurable, repeatable improvement is observed across multiple random seeds.

The remainder of this report is structured as follows: Section 2 reviews the baseline paper and the specific gap it reports; Section 4 describes the reproduced baseline and the proposed fusion architecture; Section 5 describes the synthetic telemetry generator, training pipeline, and deployment; Section 6 reports results across three seeds; Section 7 discusses limitations and future work.

2 Related Work

Cluster and data-center monitoring has traditionally relied on static threshold rules applied to individual telemetry metrics (CPU, memory, disk, network). Such reactive monitoring only flags a problem once a metric has already crossed its limit, leaving no lead time for remediation.

2.1 Baseline: Multi-Head Attention for Telemetry-Based Violation Prediction

The baseline for this project is Thapliyal Thapliyal (2026), “A Multi-Head Attention Approach for SLA Compliance Monitoring in Data Centers” (arXiv:2605.05354, accepted at the 46th IEEE ICDCS, May 2026). The paper trains a per-customer transformer in which each attention head is dedicated 1-to-1 to a single telemetry rule (power draw, temperature, humidity), predicting a None/Level-1/Level-2 violation label 30 minutes ahead from historical sensor windows. This directly targets the same problem this project addresses – proactive, rather than reactive, violation prediction – but for colocation SLA metrics rather than cluster health metrics.

Critically, the paper’s own reported results expose a specific weakness of the strict one-head-per-metric design: the power-draw head shows “a skewed, bimodal error profile ... with systematic underprediction during high-load transients, a direct consequence of instantaneous load volatility.” The authors attribute this to the architecture’s decoupled heads never exchanging information with each other, even though their own data shows power and temperature are correlated.

2.2 The Gap This Project Targets

Because each head in Thapliyal’s design is trained and read out independently, a head has no way to use another metric’s precursor signal to anticipate its own metric’s future violation. This project reproduces the strict per-head architecture as a baseline (Section 4), confirms the same underprediction pattern on synthetic cluster telemetry, and then evaluates a cross-head fusion layer that lets the per-metric head representations attend to one another before classification.

2.3 Threshold-Based Monitoring as a Second Baseline

Traditional reactive threshold monitoring (flagging a violation only once a metric has already crossed its band in the observed window) is retained as a second, non-attention baseline, consistent with how Thapliyal frames the case for proactive monitoring against the industry-standard reactive approach.

3 Methodology

3.1 Synthetic Telemetry Generator

Real multi-tenant cluster telemetry was not available, so a synthetic generator (`src/data/telemetry_si`) produces 4-metric (CPU, memory, disk, network) sequences of 15 steps: a 10-step *history* window (the only input the models see) and a 5-step *future* window (used only to compute labels, never shown to the models). This makes the task a genuine forecasting problem rather than same-window classification – a design flaw caught and fixed during this project (an earlier version computed labels from the same window the models could see, which trivially favoured the reactive baseline).

Each metric has independent None/L1/L2 thresholds. 40% of samples contain an injected “transient” burst: a fast, non-linear ramp in a randomly chosen primary metric landing mostly or fully in the future window, plus a correlated ramp in 1-2 “rider” metrics that leads the primary metric by 3-6 steps and is visible in the history window. This construction means some future violations can only be anticipated by reading a *different* metric’s precursor signal – exactly the scenario a strict per-head architecture struggles with.

3.2 Training

Both attention models are trained with Adam ($\text{lr}=0.001$) for 30 epochs on 20,000 samples per seed (80/20 train/test split, stratified on the transient flag), using a summed per-metric cross-entropy loss with inverse-frequency class weights (to counteract the dominance of the “none” class). Three random seeds (42, 43, 44) are used to regenerate the dataset and retrain both models, so results are reported as mean and standard deviation rather than a single run.

3.3 Metrics

- **Accuracy / Macro-F1:** per-metric, averaged across the 4 metrics, over the full test set.

- **Transient Violation Recall:** of the test samples that are both transient *and* have a true future violation (L1/L2) on a given metric, the fraction where the model predicts any violation at all.
- **Transient Underprediction Bias:** $\text{mean}(\text{true label} - \text{predicted label})$ over the same subset. This is the metric that operationalises the baseline paper’s own stated gap (“systematic underprediction ... during high-load transients”); a value of 0 means no systematic bias, positive values mean the model under-calls severity.

All code, results, and the trained model weights used for deployment are in `mhsa-tdl-framework/`.

4 Design Specification

4.1 Shared Backbone

Both models share a temporal encoder: a linear projection of the 4 raw metrics (CPU, memory, disk, network) into a 32-dimensional embedding, a learned positional encoding, one standard multi-head self-attention block over the 10-step history window, and a feed-forward block with residual connections and layer normalisation. This corresponds to the “Transformer encoder backbone” stage of Thapliyal’s architecture, which jointly encodes all metrics before any per-metric specialisation.

4.2 MHSA-PerHead (baseline reproduction)

A manual multi-head attention layer splits the backbone’s output channel dimension into 4 heads (one per metric). Each head computes its own attention (query = last timestep, key/value = full window) to produce a per-head context vector, which is passed to an independent 2-layer classifier predicting a 3-class violation label (None/L1/L2) for that metric only. No information is shared across heads after the split – this is the strict 1-to-1 head-to-metric mapping used in the baseline paper.

4.3 MHSA-Fused (proposed improvement)

Identical to MHSA-PerHead, with one addition: after the per-head context vectors are computed, a single-layer self-attention block (**CrossHeadFusion**) treats the 4 head vectors as a short 4-token sequence and lets them attend to each other, with a residual connection and layer norm. The fused vectors are then passed to the same per-metric classifiers. This is the minimal architectural change needed to let e.g. the CPU head’s classifier be informed by what the Network head is currently observing.

4.4 Threshold Baseline

A traditional reactive monitor: for each metric, it takes the peak value over the last 3 steps of the *history* window only (it cannot see the future window) and buckets it into None/L1/L2 using the same thresholds used to generate ground-truth labels. It represents the industry-standard practice the baseline paper positions itself against.

5 Implementation

The system is implemented in Python 3 with PyTorch. Key modules:

- `src/data/telemetry_simulator.py` – synthetic telemetry generator described in Section 3.
- `src/models/mhsa_model.py` – `SharedBackbone`, `MHSAPerHead` (baseline), `MHSAFused` (improved), and the `CrossHeadFusion` layer.
- `src/models/baseline.py` – `ThresholdBaseline`, the reactive monitoring comparison.
- `scripts/train_and_evaluate.py` – trains and evaluates all three approaches across 3 seeds, writes `results/results_per_seed.csv` and `results/results_summary.csv`, and exports the trained `MHSAFused` weights to `models/mhsa_fused.pt` for deployment.
- `src/lambda_handler/app.py` – an AWS Lambda handler that loads the exported model and runs real inference (not a mock) over telemetry records arriving on a Kinesis stream, in a 30-day rolling window per node.
- `template.yaml` – AWS SAM template provisioning the Kinesis stream and Lambda function.
- `.github/workflows/deploy.yml` – CI pipeline that runs the training/evaluation script and, on push to `main`, deploys the SAM stack to the configured AWS region using repository secrets.

Running `python scripts/train_and_evaluate.py` reproduces every number in Section 6 from scratch (no external dataset dependency, since telemetry is synthetically generated with a fixed seed).

6 Evaluation

Table 1 reports mean $\pm$ standard deviation across 5 seeds (42-46) on held-out test data. Model weight initialisation and data-loader shuffling are seeded per run (not just the synthetic data generator), so these numbers are reproducible by re-running `scripts/train_and_evaluate.py`.

Table 1: Results across 5 seeds (mean $\pm$ std)

Model	Accuracy	Macro-F1	Transient Recall	Underprediction Bias
Threshold Baseline	0.847 ± 0.002	0.313 ± 0.002	0.025 ± 0.002	1.715 ± 0.011
MHSA-PerHead (baseline)	0.873 ± 0.011	0.565 ± 0.008	0.864 ± 0.022	0.290 ± 0.076
MHSA-Fused (improved)	0.863 ± 0.014	0.560 ± 0.010	0.884 ± 0.021	0.273 ± 0.050

6.1 Experiment 1: Reactive vs. Proactive

Both attention models overwhelmingly outperform the reactive threshold baseline at anticipating violations before they occur (Transient Recall 0.86-0.88 vs. 0.025). This is structural, not a modelling artefact: the threshold baseline only ever sees the history window, so unless a violation has already started before the window ends, it cannot know one is coming. This confirms Thapliyal’s central claim – proactive, attention-based prediction meaningfully beats reactive threshold monitoring – transfers from the SLA/colocation setting to cluster telemetry.

6.2 Experiment 2: Reproducing the Baseline Paper’s Gap

The reproduced baseline (MHSA-PerHead) shows a Transient Underprediction Bias of 0.29 severity levels – i.e. when a real L1/L2 violation occurs inside a transient window, the model’s prediction is on average 0.29 levels too low. This reproduces, on cluster telemetry, the same failure mode Thapliyal reports for the power-draw head on colocation telemetry: strictly independent per-metric heads underpredict severity during volatile, correlated bursts.

6.3 Experiment 3: Cross-Head Fusion

Adding the cross-head fusion layer (MHSA-Fused) moves both of the paper’s targeted metrics in the right direction: Transient Recall rises from 0.864 to 0.884, and Underprediction Bias falls from 0.290 to 0.273 ($\sim 6\%$ relative reduction). This comes at a small cost to the metrics the paper does not specifically flag as a gap – overall Accuracy drops from 0.873 to 0.863 and Macro-F1 from 0.565 to 0.560. None of these four differences exceed one standard deviation across the 5 seeds, so the honest reading is: a real, directionally consistent, but statistically modest improvement on the targeted gap, not a decisive win.

6.4 Discussion

An earlier run of this same experiment (before model-weight initialisation and data-loader shuffling were seeded) showed a much larger apparent improvement ($\sim 20\%$ bias reduction). That result did not replicate once training randomness was properly controlled – it was luck in the random seed, not a real effect, which is itself a useful (if uncomfortable) finding: unseeded neural network comparisons in a report this size can easily produce a result that looks like a clean win but isn’t. The properly-seeded, 5-seed result reported above is the one that should be trusted.

The remaining effect is real but modest, and is consistent with the underlying mechanism being subtle: the shared backbone already partially mixes cross-metric information before the heads split, so the fusion layer is refining an existing signal rather than introducing information that was completely unavailable before. A cleaner, larger effect would likely need either a longer training budget, an explicit auxiliary loss that penalises underprediction directly (rather than a generic cross-entropy loss), or a larger, more diverse burst-pattern vocabulary in the synthetic generator. These are noted as future work in Section 7 rather than pursued further here, given the intentionally small scope of this benchmark.

The threshold baseline’s near-zero transient recall should not be read as “threshold monitoring is bad” in general – it is doing exactly what reactive monitoring is designed to do (report current state), and is being evaluated here specifically on a forecasting task it was never meant to solve.

7 Conclusion and Future Work

This project asked whether letting per-metric attention heads exchange information via a cross-head fusion layer reduces the systematic underprediction during high-load transients reported by Thapliyal (2026) for a strict one-head-per-metric SLA monitoring architecture. Reproducing that architecture on synthetic cluster telemetry confirmed the same failure mode (underprediction bias of 0.29 severity levels on transient windows), and the proposed fusion layer moved both targeted metrics in the right direction (recall $0.864 \rightarrow 0.884$, bias $0.290 \rightarrow 0.273$) across 5 properly-seeded training runs, at a small cost to overall accuracy ($0.873 \rightarrow 0.863$). Both attention-based models substantially outperformed reactive threshold monitoring at anticipating violations ahead of time (86-88% vs. 2.5% recall), confirming the baseline paper’s central premise transfers to cluster health telemetry.

The improvement from fusion is real but modest, and none of the differences exceed one standard deviation across seeds – an honest limitation rather than a negative result. A first, unseeded version of this experiment showed a much larger apparent effect that turned out to be an artefact of uncontrolled random initialisation rather than the fusion layer itself; this was caught by re-running with torch’s RNG properly seeded, which is itself a reminder that small neural-network comparisons need seed control to be trustworthy.

Future work:

- Evaluate on real cluster telemetry (e.g. the Google Cluster Trace dataset) rather than synthetic data, to check whether the fusion improvement holds under real, non-stationary failure patterns.
- Replace the generic cross-entropy loss with an asymmetric loss that penalises underprediction more than overprediction, directly targeting the failure mode this project measures.
- Extend the burst-pattern vocabulary in the synthetic generator to a wider range of cross-metric lead/lag relationships, to test whether the fusion benefit scales with how much genuinely cross-metric information the task requires.

References

Thapliyal, O. (2026). A multi-head attention approach for SLA compliance monitoring in data centers, *arXiv preprint arXiv:2605.05354*, accepted at the 46th IEEE International Conference on Distributed Computing Systems (ICDCS) .
URL: <https://arxiv.org/abs/2605.05354>